Reviewer Pack — Bounded Mean-to-Max Ratio for Tropical Doubling Defect at β=…

Entry c18fc45a5b3a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 09:57:53 UTC

Bounded Mean-to-Max Ratio for Tropical Doubling Defect at β=5

Entry ID: `c18fc45a5b3a` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-29 09:57:53 UTC

1. Conjecture

Field A (mathematical branch): Discrepancy theory **Field B** (complexity object): Tropical Fourier analysis

Statement:

> Let f:Z_n→R be a tropical polynomial with coefficients drawn iid Uniform[0,1], let g = tropical self-convolution of f under the min-plus semiring, and let MFC_β(h) := min_{k≠0} |Σ_x exp(-β·h(x))·ω^{kx}| (β=5 Maslov dequantization, ω=exp(2πi/n)) and Δ(f) := |MFC_β(g) − 2·MFC_β(f)|. Conjecture: for every n ∈ {8,12,16,20,24,28,32}, sampling 200 polynomials per n with seeds 1..200, the ratio R(n) := max_f Δ(f) / mean_f Δ(f) satisfies R(n) ≤ 3 + log_2(n)/4. A single n where R(n) > 3 + log_2(n)/4 over the seeds 1..200 falsifies the conjecture.

Rationale (proposer's reasoning):

> SC3 asserts the worst-case Δ is not pathological compared to the mean; to advance the program we need a quantitative envelope. A logarithmic envelope is the natural sub-Gaussian guess from concentration heuristics: each Δ is a Lipschitz function of n iid Uniform[0,1] coefficients, so max/mean should grow like √log n at worst — the 3+log_2(n)/4 bound is intentionally generous yet still defeats any heavy-tailed pathology.

Taxonomy category: `tropical_discrepancy_finite_beta` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `46239e2cf89b7d6d`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> For each n∈{8,12,16,20,24,28,32}, draw 200 tropical polynomials (seeds 1..200, Uniform[0,1]) and compute R(n)=max_f Δ(f)/mean_f Δ(f) with Δ via Maslov DFT at β=5. SUPPORTED iff R(n) ≤ 3+log_2(n)/4 for all 7 values of n; FALSIFIED if any single n exceeds its envelope.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.97	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.92	SAFE	SAFE
KARP_LIPTON	SAFE	0.99	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `tropical polynomial min-plus convolution Fourier discrepancy Maslov dequantization` - `tropical Fourier transform doubling defect mean-to-max ratio random coefficients` - `min-plus semiring self-convolution discrepancy bound exponential sum cyclic group`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=6.4s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find max pivot in column i
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        # Swap rows
        A[i], A[max_row] = A[max_row], A[i]
        # Eliminate column i
        pivot = A[i][i]
        for j in range(i, n):
            A[i][j] /= pivot
        for j in range(n):
            if j != i:
                factor = A[j][i]
                for k in range(i, n):
                    A[j][k] -= factor * A[i][k]

def matrix_multiply(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[0] * p for _ in range(m)]
    for i in range(m):
        for j in range(p):
```

```
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def min_plus_convolution(f, g, n):
    result = [0] * (2*n - 1)
    for i in range(n):
        for j in range(n):
            result[(i + j) % (2*n - 1)] += f[i] + g[j]
    return result

def maslov_dft(f, beta, n):
    omega = math.exp(2 * math.pi / n)
    mfc = [0] * n
    for k in range(n):
        sum_val = 0
        for x in range(n):
            sum_val += math.exp(-beta * f[x]) * (omega ** (k * x))
        mfc[k] = abs(sum_val)
    return min(mfc)

def delta(f, g, beta, n):
    mfc_f = maslov_dft(f, beta, n)
    mfc_g = maslov_dft(g, beta, n)
    return abs(mfc_g - 2 * mfc_f)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [8, 12, 16, 20, 24, 28, 32]
    results = []

    for n in n_values:
        deltas = []
        for _ in range(200):
            f = [random.random() for _ in range(n)]
            g = min_plus_convolution(f, f, n)
            delta_value = delta(f, g, 5, n)
            deltas.append(delta_value)

        mean_delta = sum(deltas) / len(deltas)
        max_delta = max(deltas)
        R_n = max_delta / mean_delta
        bound = 3 + math.log2(n) / 4

        results.append({
            "n": n,
            "mean_delta": mean_delta,
            "max_delta": max_delta,
            "R_n": R_n,
            "bound": bound
        })

    if any(result["R_n"] > result["bound"] for result in results):
        conjecture_holds = False
        counterexample = f"n={result['n']}, R(n)={result['R_n']}, bound={result['bound']}"
    else:
```

```
        conjecture_holds = True
        counterexample = ""

    return {
        "metric_name": "R(n)",
        "metric_value": sum(result["R_n"] for result in results) / len(results),
        "instances_tested": 200 * len(n_values),
        "n_max": max(n_values),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_

    mean_R_n = sum(result["R_n"] for result in results) / len(results)
    std_R_n = (sum((result["R_n"] - mean_R_n) ** 2 for result in results) / len(results)) ** 0.5
    support_fraction = sum(1 for result in results if result["R_n"] <= result["bound"]) / len(resu

    print(f"RESULT: {'SUPPORTED' if support_fraction >= 0.8 else 'FALSIFIED'} mean={mean_R_n:.6f}
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
48865, "instances_tested": 1400, "n_max": 32, "conjecture_holds": True, "counterexample": ""}
TRIAL: {"seed": 503, "metric_name": "R(n)", "metric_value": 1.879204, "instances_tested": 1400, "n_ma
TRIAL: {"seed": 547, "metric_name": "R(n)", "metric_value": 1.862319, "instances_tested": 1400, "n_ma
TRIAL: {"seed": 593, "metric_name": "R(n)", "metric_value": 1.919835, "instances_tested": 1400, "n_ma
TRIAL: {"seed": 631, "metric_name": "R(n)", "metric_value": 1.931334, "instances_tested": 1400, "n_ma
TRIAL: {"seed": 677, "metric_name": "R(n)", "metric_value": 1.865005, "instances_tested": 1400, "n_ma
TRIAL: {"seed": 727, "metric_name": "R(n)", "metric_value": 1.953283, "instances_tested": 1400, "n_ma
TRIAL: {"seed": 773, "metric_name": "R(n)", "metric_value": 1.968555, "instances_tested": 1400, "n_ma
TRIAL: {"seed": 821, "metric_name": "R(n)", "metric_value": 2.030123, "instances_tested": 1400, "n_ma
TRIAL: {"seed": 877, "metric_name": "R(n)", "metric_value": 1.959739, "instances_tested": 1400, "n_ma
TRIAL: {"seed": 929, "metric_name": "R(n)", "metric_value": 1.887390, "instances_tested": 1400, "n_ma

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_88cf8e27.py", line 119, in <module>
    mean_R_n = sum(result["R_n"] for result in results) / len(results)
                                                ^^^^^^^
NameError: name 'results' is not defined. Did you mean: 'result'?
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

The test script crashed with a NameError ('results' is not defined') before aggregating per-n R(n) values, so no RESULT line confirms the pre-registered support condition across all 7 n-values. Per-trial outputs show R(n) values under 2, but the required aggregate check and bootstrap CI were never computed. | next: Fix the NameError (define/collect 'results' list) and re-run the full sweep so per-n R(n) and bootstrap 95% CI upper bounds can be checked against 3 + log2(n)/4.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	33641
2	preregistration	claude_max	opus	0	0	6486
3	novelty	claude_max	opus	0	0	3422
4	novelty	claude_max	opus	0	0	6420
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11608
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10607
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10623
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14748
9	judge	claude_max	opus	0	0	5631

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 103184 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/c18fc45a5b3a.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c18fc45a5b3a.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c18fc45a5b3a.tar.gz` (if generated)